



AI VIET NAM – AI Conquer 2026

A Comparative Study of Tree-based Models for Healthcare Risk Prediction: Cross-validation and SHAP-based Stability Analysis

Group Numby

Name	Position	Role
Le Quang Thanh	Tech Leader	Team Leader
Nguyen Van Vi	AI Engineer (Pipeline)	Member
Le Thanh Lam	AI Engineer (Data)	Member
Pham Minh Dang Tran	AI Engineer (Model)	Member
Tan Du	QA / Reviewer	Member

I. Research Summary

Contents

I.	Research Summary	1
II.	Introduction and Research Questions	4
II.1.	Research Context	4
II.2.	Research Problem	4
II.3.	Research Questions	5
II.4.	Contributions	5
III.	Related Work	5
III.1.	Boosting-Based Tree Ensembles	5
III.2.	AdaBoost	6
III.3.	XGBoost	6
III.4.	LightGBM	6
III.5.	Explainability and Stability of Tree Models	6
III.6.	Research Gap and Motivation	6
IV.	Methodology and Research Architecture	7
V.	Data and Experimental Design	7
V.1.	Dataset Description	7
V.2.	Exploratory Data Analysis	9
V.3.	Data Preprocessing Pipeline	11
V.4.	Model Experimental Design	13
V.5.	Model Configuration and Hyperparameters	14
V.6.	Evaluation Metrics and Model Comparison	14
V.7.	SHAP Explainability and Feature-Ranking Stability (RQ3)	15
VI.	Results and Discussion	16
VI.1.	Predictive Performance Across Datasets (RQ1)	16
VI.2.	Predictive Stability Across Cross-Validation Folds (RQ2)	17
VI.3.	SHAP Feature-Importance Stability Across Cross-Validation Folds (RQ3)	18
VI.4.	Discussion	19
VII.	Conclusion and Future Work	20

VII.1. Conclusion	20
VIII. References	21
Appendix	22

II. Introduction and Research Questions

II.1. Research Context

Healthcare risk prediction has become an important application of machine learning, particularly with the increasing availability of structured healthcare data. Machine learning models can be used to identify patterns in demographic, clinical, and physiological features and estimate the risk of developing a disease or experiencing an adverse health outcome. Such predictive models may provide useful decision-support information for healthcare-related analysis.

Among machine learning approaches for structured data, tree-based models are widely used because they can capture nonlinear relationships and feature interactions while requiring relatively few assumptions about the underlying data distribution. In particular, boosting-based models such as AdaBoost, XGBoost, and LightGBM have demonstrated strong predictive capabilities and are commonly applied to classification problems.

However, healthcare datasets can differ substantially in terms of sample size, feature distributions, missing values, and class imbalance. As a result, the performance of a particular model may vary across datasets. Evaluating multiple models across multiple healthcare datasets under a consistent experimental protocol can therefore provide a more comprehensive understanding of their behavior.

II.2. Research Problem

Although tree-based models can achieve strong predictive performance, comparing models based solely on a single train-test split or a single performance metric may provide an incomplete view of their reliability. A model that performs well on one particular split may exhibit considerable variation when trained on different subsets of the data.

Cross-validation provides a systematic approach for evaluating this variability by repeatedly training and evaluating models on different data partitions. Therefore, model comparison should consider not only average predictive performance but also the consistency of performance across cross-validation folds.

Furthermore, predictive performance does not indicate why a model produces its predictions. In healthcare-related machine learning, identifying influential features can provide additional insight into model behavior. SHapley Additive exPlanations (SHAP) can be used to quantify the contribution of individual features to model predictions. However, feature importance obtained from a single training run may not necessarily remain consistent when the training data changes.

This raises a second stability concern: whether the explanations produced by a model remain consistent across different cross-validation folds. A feature that appears highly important in one fold but substantially decreases in importance in other folds may indicate that the corresponding interpretation is sensitive to the training data.

Therefore, this study considers both predictive stability and explanation stability when comparing tree-based models for healthcare risk prediction.

II.3. Research Questions

Based on the research problem described above, this study investigates the following research questions:

- **RQ1:** How do AdaBoost, XGBoost, and LightGBM compare in healthcare risk prediction across multiple datasets?
- **RQ2:** How stable is the predictive performance of these models across cross-validation folds?
- **RQ3:** How stable are SHAP-based feature importance rankings across cross-validation folds?

II.4. Contributions

This study makes the following contributions:

- **Systematic comparison of boosting-based tree models.** We provide a controlled empirical comparison of AdaBoost, XGBoost, and LightGBM for healthcare risk prediction across multiple datasets using a consistent experimental protocol.
- **Evaluation of predictive stability.** In addition to reporting predictive performance, we analyze the variability of model performance across cross-validation folds to assess the consistency of the models.
- **Analysis of SHAP explanation stability.** We investigate the stability of SHAP-based feature importance rankings across cross-validation folds, providing an empirical assessment of whether model explanations remain consistent under different training subsets.
- **Multi-dataset evaluation.** By evaluating the models across multiple healthcare datasets, the study examines whether observed performance and explanation patterns are consistent across different data characteristics.
- **Reproducible experimental framework.** The experiments are conducted using a standardized and reproducible pipeline, enabling consistent comparison across models, datasets, and cross-validation folds.

III. Related Work

III.1. Boosting-Based Tree Ensembles

Boosting constructs an ensemble sequentially so that later learners focus on errors made by earlier learners. This makes boosting well suited to structured tabular prediction because non-linear effects and feature interactions can be captured without requiring a single highly complex tree. The three models used in this study represent different generations of boosting-based tree

methods: AdaBoost provides a classical boosting baseline, while XGBoost and LightGBM are modern gradient-boosted decision-tree systems designed for stronger predictive performance and computational efficiency.

III.2. AdaBoost

AdaBoost is one of the foundational boosting algorithms. It updates observation weights after each boosting round, increasing emphasis on samples that were previously misclassified, and combines the weak learners into a weighted final classifier [1]. In this study, AdaBoost serves as the classical baseline and uses decision stumps as weak learners. Keeping the base learners simple provides a clear comparison with the deeper and more heavily regularized gradient-boosting systems used by XGBoost and LightGBM.

III.3. XGBoost

XGBoost extends gradient tree boosting with both statistical regularization and systems-level optimizations for scalable learning. Its design includes a regularized objective, sparsity-aware split handling, approximate tree learning, and efficient parallel computation [2]. It also exposes controls such as tree depth, shrinkage, row subsampling, column subsampling, and regularization. These controls are useful for the present study because the three healthcare datasets differ substantially in sample size, dimensionality, and class prevalence, while the same fixed model configuration must remain reproducible across datasets.

III.4. LightGBM

LightGBM is a gradient-boosted decision-tree framework designed for high computational efficiency. Ke et al. introduced Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB) to reduce computational cost while preserving predictive quality on large and high-dimensional datasets [3]. LightGBM also grows trees leaf-wise, allowing it to fit complex nonlinear relationships efficiently. Because leaf-wise growth can increase model complexity, parameters such as the number of leaves, maximum depth, minimum child samples, learning rate, subsampling, and regularization are controlled explicitly in the experimental configuration.

III.5. Explainability and Stability of Tree Models

Predictive performance alone does not show which variables drive model outputs. SHAP (SHapley Additive exPlanations) provides additive feature attributions for individual predictions and establishes desirable consistency properties for feature attribution [4]. Tree-specific SHAP algorithms make these explanations computationally practical for tree ensembles, and prior work has demonstrated their use in medical prediction settings [5]. This motivates evaluating not only predictive performance across cross-validation folds but also whether feature-importance rankings remain consistent as the training sample changes.

III.6. Research Gap and Motivation

The goal of this study is therefore broader than identifying a single model with the highest average score. AdaBoost, XGBoost, and LightGBM are evaluated under a common preprocessing, cross-

validation, metric, and hold-out testing protocol across multiple healthcare datasets. The study then examines two forms of stability: fold-to-fold variation in predictive performance and fold-to-fold variation in SHAP-based feature rankings. This design supports the three research questions by separating average predictive strength from sensitivity to the particular training sample and from the stability of model explanations.

IV. Methodology and Research Architecture

V. Data and Experimental Design

V.1. Dataset Description

Three publicly available Kaggle datasets are used, all framed around the same binary target: whether a respondent or patient has (or has had) heart disease or a heart attack. The datasets differ substantially in size, feature type, and data-quality characteristics, which is intentional: comparing AdaBoost, XGBoost, and LightGBM across three heterogeneous datasets, rather than a single one, lets RQ1–RQ3 be answered under different sample sizes, feature compositions, and class-imbalance regimes rather than for one dataset in isolation. Each dataset is loaded directly from its raw CSV file under `data/raw/` via a single shared `load_raw_dataset()` function, with no dataset-specific loading logic.

Dataset	Rows	Numeric	Categorical	Missing	Positive rate
Dataset 1	445,132	6	34	38/40 columns	5.64%
Dataset 2	918	6	5	none	55.34%
Dataset 3	253,680	7	14	none	9.42%

Table 1: Raw shape, feature-type composition, missingness, and target positive rate for the three datasets before any preprocessing.

Dataset 1 — Personal Key Indicators of Heart Disease (2022): sourced from the Kaggle 2022 BRFSS release [6] and distributed as `heart_2022_with_nans.csv`. This variant is used deliberately, rather than the pre-cleaned no-NaN release, so that the preprocessing pipeline demonstrates a realistic missing-value-handling workflow. The raw table has 445,132 rows and 40 columns: 34 categorical columns (`State`, `Sex`, `GeneralHealth`, and most of the Yes/No health-history flags such as `HadHeartAttack`, `HadDiabetes`, `HadStroke`) and 6 numeric columns (`PhysicalHealthDays`, `MentalHealthDays`, `SleepHours`, `HeightInMeters`, `WeightInKilograms`, `BMI`). The target column, `HadHeartAttack`, is a Yes/No self-report that is mapped to a binary 0/1 label during preprocessing; before cleaning, 93.67% of respondents (416,959) answered "No", 5.64% (25,108) answered "Yes", and 0.69% (3,065) left the field blank, giving a roughly 17:1 imbalance between the majority and minority classes once the missing-target rows are dropped.

Thirty-eight of the 40 columns contain at least one missing value; only `State` and `Sex` are fully populated. Table 2 lists the six most-affected columns, all vaccination/screening-history fields

that respondents frequently skip or do not know the answer to. The core numeric health metrics have moderate missingness (**BMI** 10.96%, **WeightInKilograms** 9.45%, **HeightInMeters** 6.44%), while the disease-history Yes/No flags and core demographics are the most complete, at well under 1% missing — a pattern typical of self-reported survey data, where harder-to-recall or less commonly asked questions accumulate more gaps.

Column	Missing rate
TetanusLast10Tdap	18.54%
PneumoVaxEver	17.31%
HIVTesting	14.86%
ChestScan	12.59%
CovidPos	11.40%
HighRiskLastYear	11.37%

Table 2: The six most-missing columns in Dataset 1 (raw data, before imputation).

Dataset 2 — Heart Failure Prediction: sourced from the fedesoriano Kaggle release [7] and distributed as `heart.csv`. This is the smallest and already mostly-clean table used in this study: 918 rows and 12 columns, combining 5 clinical/demographic categorical columns (**Sex**, **ChestPainType**, **RestingECG**, **ExerciseAngina**, **ST_Slope**) with 6 numeric columns (**Age**, **RestingBP**, **Cholesterol**, **FastingBS**, **MaxHR**, **Oldpeak**) and the pre-encoded binary target **HeartDisease**. There are zero missing values across all 918 rows, so the dataset appears complete on the surface, but two of its numeric columns use 0 as an undocumented placeholder for "not measured" rather than a genuine reading: exactly 1 row has **RestingBP** == 0 and 172 rows (18.7%) have **Cholesterol** == 0, far too many to be real zero-cholesterol readings given that a cholesterol or blood-pressure value of 0 is not physiologically possible. **DatasetConfig.zero_as_missing_columns** records both columns so that the preprocessing pipeline recodes these placeholder zeros as NaN before imputation rather than taking them at face value. Unlike Dataset 1, the target here is close to balanced: 508 patients (55.34%) have heart disease and 410 (44.66%) do not, which is typical of a clinical dataset curated specifically for a heart-disease classification task rather than drawn from a general population survey.

Dataset 3 — Heart Disease Health Indicators (BRFSS 2015): sourced from the alexteboul Kaggle release [8] and distributed as `heart_disease_health_indicators_BRFSS2015.csv`. The raw table has 253,680 rows and 22 columns, all already numerically encoded as `float64` – including conceptually binary flags (**HighBP**, **Smoker**, **Sex**) and small ordinal scales (**GenHlth** 1–5, **Age** 1–13 brackets, **Education** 1–6, **Income** 1–8). Seven columns (**BMI**, **GenHlth**, **MentHlth**, **PhysHlth**, **Age**, **Education**, **Income**) are treated as numeric and the remaining 14 as categorical. There are zero missing values anywhere in the dataset, making it the cleanest of the three in terms of completeness. However, 23,899 rows (9.42%) are exact duplicates of another row across all 22 columns – plausible here because every feature is binary or a small discrete scale, so two distinct respondents can easily share an identical combination of answers. This is a data-quality issue distinct from missingness, and **DatasetConfig** flags Dataset 3 with `drop_duplicate_rows=True` so these rows are removed before the target is cleaned (Section V.3.). The target, **HeartDiseaseorAttack**, is heavily imbalanced on the raw data: 229,787 respondents (90.58%) report no heart disease/attack history and 23,893 (9.42%) do, a roughly 1:9.6 minority-to-majority ratio, similar in shape to Dataset 1 but somewhat less extreme.

Across all three datasets, the target definition, raw imbalance, and data-quality quirks summarized above directly motivate the dataset-specific preprocessing decisions described in Section V.3.: the class-imbalance policy (F1), the zero-as-missing recoding restricted to Dataset 2, and the duplicate-row removal restricted to Dataset 3.

V.2. Exploratory Data Analysis

Before any preprocessing decision is made, each of the three raw datasets is examined independently in `notebooks/01_eda.ipynb` to characterize its class balance, missingness, numeric-feature relationships, and the strength of association between individual features and the target. The purpose of this stage is not to fit any model, but to surface the data-quality issues and predictive signals that later motivate specific preprocessing choices (see Section V.3.) and the class-imbalance policy adopted in the modeling stage (Section V.1. and finding F1 of the project’s scope review).

Class imbalance. All three datasets exhibit some degree of class imbalance, but to very different extents. Dataset 1 is the most imbalanced, with a raw positive rate of 5.64%, giving balanced class weights of approximately $\{0 : 0.53, 1 : 8.80\}$ – a positive example is weighted roughly $16.6\times$ more than a negative one under scikit-learn’s balanced weighting scheme [9]. Dataset 3 is imbalanced in the same direction but less severely, with a raw positive rate of 9.42% and balanced weights of approximately $\{0 : 0.55, 1 : 5.31\}$ (about $9.6\times$). Dataset 2 is the outlier among the three: its positive rate of 55.34% is close to balanced, giving weights near 1.0 in both directions ($\{0 : 1.12, 1 : 0.90\}$) and requiring very little correction. This spread of imbalance regimes across the three datasets is precisely what motivates RQ1/RQ2’s design: a class-imbalance policy that only worked well on a near-balanced dataset like Dataset 2 would not generalize to the 17:1 imbalance of Dataset 1.

Missingness. Dataset 1 is the only one of the three with meaningful missing data: 38 of its 40 columns contain at least one missing value, led by vaccination/screening-history fields that respondents often skip (Table 2). Datasets 2 and 3 report zero missing values on the surface, but Dataset 2’s completeness is misleading – its `RestingBP` and `Cholesterol` columns use an undocumented 0 placeholder for "not measured" (1 and 172 rows respectively) that is not distinguishable from a genuine reading without domain knowledge. This is why the EDA stage treats "zero missing values" and "no data-quality issues" as two separate questions rather than assuming one implies the other.

Numeric feature correlations. Correlation matrices computed over each dataset’s numeric columns show no evidence of problematic multicollinearity in any of the three datasets, though each has its own notable pairwise relationships. In Dataset 1, `WeightInKilograms` and `BMI` are strongly correlated ($r = 0.86$), which is expected since BMI is mathematically derived from weight and height; `HeightInMeters` and `WeightInKilograms` are moderately correlated ($r = 0.47$), while `HeightInMeters` and `BMI` are nearly uncorrelated ($r = -0.03$), confirming that BMI successfully normalizes out the effect of height. `PhysicalHealthDays` and `MentalHealthDays` are moderately correlated ($r = 0.32$). In Dataset 2, no pair of numeric features is strongly correlated;

the strongest relationships are **Age** $\leftrightarrow$ **MaxHR** ($r = -0.38$) and **Age** $\leftrightarrow$ **Oldpeak** ($r = 0.26$), with **Cholesterol** weakly negatively correlated with **FastingBS** ($r = -0.26$), plausibly diluted by the placeholder-zero **Cholesterol** values noted above. In Dataset 3, **PhysHlth** and **GenHlth** are the most correlated pair ($r = 0.52$), **MentHlth** and **PhysHlth** are moderately correlated ($r = 0.35$), and **Education** and **Income** show the strongest socioeconomic correlation overall ($r = 0.45$). In all three cases, these pairwise relationships are informative but well below the 0.9 correlation threshold later used for feature reduction (Section V.3.), so they surface as candidates for the modeling stage rather than as columns to drop outright at the EDA stage.

Outlier detection (Dataset 2 only). Dataset 2 is the only dataset whose numeric columns are genuine continuous clinical measurements rather than mostly binary or discrete survey answers, which is why IQR (Tukey-fence) outlier detection is examined here and later applied only to Dataset 2 during preprocessing (finding F5). Computed on the full raw dataset for exploratory purposes, **Age** has no values outside its fence; **RestingBP** (bounds 90–170) flags 27 potential outliers; **Cholesterol** (bounds 105.6–376.6, computed after treating its zero-placeholders as missing) flags 23; **MaxHR** (bounds 66–210) flags 2; and **Oldpeak** (bounds -2.25 – 3.75) flags 16, consistent with its long right tail. These full-dataset bounds are for exploratory characterization only; the bounds actually used to clip Dataset 2’s train/test splits in Section V.3. are refit on the training split alone to avoid leaking test-set extremes into the fences.

Target rate by category. Categorical features associated with self-reported general health status are, across every dataset that has one, by far the strongest single predictors observed during EDA. In Dataset 1, the heart-attack rate rises monotonically with self-reported **GeneralHealth** from 1.53% ("Excellent") to 2.84% ("Very good"), 5.78% ("Good"), 11.99% ("Fair"), and 21.08% ("Poor") – a roughly 14-fold spread across five categories, in contrast to BMI, whose distribution is nearly identical between positive and negative cases (median around 27–28 in both groups) and is therefore a comparatively weak discriminator on its own. In Dataset 2, both categorical predictors examined are strong: patients with asymptomatic chest pain (**ASY**) have a heart-disease rate of 79.03%, far higher than **ATA** (13.87%), **NAP** (35.47%), or **TA** (43.48%) – counterintuitively, the absence of typical chest-pain symptoms is the strongest single warning sign in this dataset – and a **Flat** (82.83%) or **Down** (77.78%) ST-segment slope is associated with a much higher rate than an **Up**-sloping segment (19.75%). In Dataset 3, the same **GenHlth** pattern recurs almost identically to Dataset 1 despite the two datasets being independent BRFSS survey years: the rate rises from 2.24% ("Excellent") to 4.63%, 10.46%, 21.31%, and 34.00% ("Poor"); **HighBP** is also strongly associated, with 16.47% of respondents with high blood pressure reporting heart disease/attack versus only 4.12% of those without.

Taken together, these EDA findings motivate three concrete preprocessing decisions detailed in Section V.3.: a shared class-imbalance policy applied at model-fit time rather than by resampling (F1, since the degree of imbalance varies too much across datasets for a single resampling ratio to be appropriate); IQR-based outlier clipping scoped to Dataset 2 only (F5, since Datasets 1 and 3 are mostly binary/discrete survey data where a genuinely high value is itself a disease signal rather than noise); and dataset-specific data-quality handling – zero-as-missing recoding for Dataset 2 and duplicate-row removal for Dataset 3 – rather than a single one-size-fits-all cleaning rule.

V.3. Data Preprocessing Pipeline

All three datasets are processed by the same ordered pipeline (`src/main.py`'s `process_dataset()`, backed by `src/data/loader.py`, `src/data/preprocessing.py`, and `src/data/split.py`), configured per dataset through the `DatasetConfig` registry in `src/config.py`. Every fitted statistic used anywhere in the pipeline – imputation values, encoders, the scaler, outlier bounds, and the feature-reduction selector – is fit on the training split only and then reused, unmodified, to transform the test split, so that no information from the held-out test rows leaks into any fitted object. The pipeline runs in the following order for each dataset.

- 1. Load and normalize.** `load_raw_dataset()` reads the raw CSV, strips whitespace from column names, and drops any fully-empty columns; this step is identical for all three datasets.
- 2. Duplicate removal (Dataset 3 only).** Datasets whose `DatasetConfig.drop_duplicate_rows` flag is set have exact-duplicate rows removed before any further cleaning. Only Dataset 3 opts in: this drops 23,899 of its 253,680 rows (9.42%), leaving 229,781 rows.
- 3. Zero-as-missing recoding (Dataset 2 only).** Columns listed in `DatasetConfig.zero_as_missing_columns` have their 0 values converted to NaN before imputation. Only Dataset 2 opts in, for `Cholesterol` (172 rows) and `RestingBP` (1 row), since a 0 reading is not a physiologically plausible measurement for either field.
- 4. Target cleaning.** `clean_target()` drops any row with a missing target value and casts the target to a binary integer. For Dataset 1 this drops 3,065 rows (0.69%), leaving 442,067 rows (416,959 negative / 25,108 positive). Datasets 2 and 3 have no missing target values, so this step only performs the type cast; for Dataset 3, applied after duplicate removal, the resulting split is 206,064 negative / 23,717 positive rows (89.68% / 10.32%), slightly less imbalanced than the pre-duplicate-removal figures reported in Section V.1. since a disproportionate share of the dropped duplicate rows were negative.
- 5. Class-imbalance-policy computation (informational only).** `compute_class_weights()` computes scikit-learn "balanced" class weights from the cleaned target column of each dataset (Dataset 1: {0 : 0.53, 1 : 8.80}; Dataset 2: {0 : 1.12, 1 : 0.90}; Dataset 3: {0 : 0.56, 1 : 4.84}). Per the project's class-imbalance policy (finding F1 of the scope review), these weights are not used to resample or drop rows at the preprocessing stage; they are reported here for traceability and passed as `sample_weight` at model-fit time in the modeling stage described later in this section, so that the reported cross-validation and test metrics still reflect each dataset's natural class prevalence.
- 6. Explicit column typing.** `infer_column_types()` splits the remaining feature columns into numeric and categorical using each dataset's `DatasetConfig.numeric_columns` as an explicit allow-list, rather than inferring types from pandas dtypes. This matters most for Dataset 3, whose columns are all stored as `float64` regardless of whether they are conceptually binary flags, small ordinal scales, or continuous measurements – dtype-based inference would have misclassified most of its columns.
- 7. Stratified train/test split.** `split_train_test()` performs an 80/20 stratified split of each dataset, preserving each dataset's positive rate in both partitions, before any imputer, encoder, scaler, outlier bound, or feature selector is fit. The held-out test partition is not touched again

until final model evaluation. This gives 353,653 / 88,414 rows for Dataset 1, 734 / 184 rows for Dataset 2, and 183,824 / 45,957 rows for Dataset 3 (train / test).

8. Outlier clipping (Dataset 2 only). For datasets with `iqr_outlier_columns` configured, `fit_outlier_bounds()` computes Tukey-fence ($\text{IQR} \times 1.5$) bounds from the training split only, and `apply_outlier_bounds()` clips (rather than drops) values outside those bounds in both splits. Only Dataset 2 opts in, on `Age`, `RestingBP`, `Cholesterol`, `MaxHR`, and `Oldpeak` (its binary-valued `FastingBS` column is deliberately excluded, since IQR bounds on a 0/1 flag would collapse to a single point and clip away the entire minority signal). Fit on the 734-row training split, `Age` has no values outside its fence, while `RestingBP` (bounds 88.50–172.50), `Cholesterol` (bounds 109.50–377.50, computed after the zero-as-missing recoding above), `MaxHR` (bounds 63.50–211.50), and `Oldpeak` (bounds -2.25 – 3.75) flag 20, 20, 2, and 13 training values respectively for clipping. These train-only bounds are close to, but not identical to, the full-dataset bounds reported during EDA (Section V.2.), which is the expected effect of fitting on 80% of the data rather than 100%.

9. Impute, encode, and scale (fit on train only). `fit_preprocessor()` fits a median imputer for numeric columns and a mode imputer for categorical columns, label-encodes binary categorical columns to 0/1, one-hot encodes multi-category columns, and standardizes numeric columns to zero mean/unit variance, all fit on the training split only; `transform_features()` then applies those exact fitted objects to the test split. This yields intermediate processed shapes of $353,653 \times 133$ / $88,414 \times 133$ for Dataset 1 (22 binary-encoded and 11 one-hot-encoded categorical columns, 6 standardized numeric columns), 734×19 / 184×19 for Dataset 2 (2 binary, 3 one-hot, 6 numeric), and $183,824 \times 24$ / $45,957 \times 24$ for Dataset 3 (13 binary, 1 one-hot on `Diabetes`, 7 numeric/ordinal).

10. Feature reduction (fit on processed train only). `fit_feature_selector()` drops near-constant columns (variance below 10^{-4}) and, among pairs of columns correlated above 0.9, the member of the pair contributing less information, fit on the processed training features only and then applied identically to the test split via `apply_feature_selector()`. No dataset has any near-constant column. Two datasets have a highly-correlated one-hot column dropped: Dataset 1 drops `CovidPos_Yes` ($r \approx 0.93$ with `CovidPos_No`, since the raw three-category `CovidPos` column is dominated by two of its three categories), and Dataset 3 drops `Diabetes_2.0` (correlated with `Diabetes_0.0` for the same dominant-plus-rare-category reason). Dataset 2 drops nothing, since none of its one-hot columns exceed the threshold on the 734-row training split – a useful negative result showing that feature reduction is applied uniformly but only actually removes columns where the training data calls for it. Table 3 summarizes the final shapes written to disk.

Dataset	Train rows	Test rows	Final features (excl. target)
Dataset 1	353,653	88,414	131
Dataset 2	734	184	18
Dataset 3	183,824	45,957	22

Table 3: Final processed train/test row counts and feature counts, after feature reduction, for the three datasets.

11. Persist train/test files. The final processed train and test data frames are written to `data/processed/<dataset>/train.csv` and `test.csv` with 6 decimal places of floating-point

precision – more than sufficient for a standardized z -score, while keeping file sizes a fraction of full `float64` output.

12. Stratified 5-fold index generation. Independently of the held-out test split above, `stratified_kfold_splits()` partitions each dataset’s processed training rows into $N_SPLITS = 5$ stratified folds using scikit-learn’s `StratifiedKFold` with shuffling and the project’s fixed random seed (`RANDOM_STATE = 42`); for every fold, 4/5 of the training rows serve as that fold’s training data and the remaining 1/5 serves as its validation data, rotated across all 5 folds. For every dataset, each fold’s validation positive rate stays close to that dataset’s overall training positive rate (5.68% for Dataset 1, 55.31% for Dataset 2, 10.32% for Dataset 3), confirming that stratification is preserved at the fold level and not only across the original train/test split. The resulting fold assignment is written row-for-row, in the same row order as `train.csv`, to `data/processed/<dataset>/kfold_indices.csv`, so the two files can be joined purely by row position. `config.yaml` points each dataset’s `kfold_path` at this same file, so that the model-evaluation and SHAP stability analyses described later in this section (particularly the SHAP methodology in Section V.7.) operate on identical, reproducible folds across all three candidate algorithms – a prerequisite for the paired statistical comparison used to answer RQ2 and RQ3.

The pipeline is covered by a dedicated `pytest` suite that exercises each of these decisions directly against the data used in this study: outlier-bound fitting and clipping semantics (`tests/test_outliers.py`), the train-only feature-reduction policy (`tests/test_feature_reduction.py`), and the balanced-sample-weight class-imbalance policy (`tests/test_imbalance.py`). Together with the leakage-avoidance pattern applied consistently at every fitting step above, this test coverage is what lets the reproducible experimental framework claimed in the Introduction extend all the way down to the data layer, not just the modeling layer.

V.4. Model Experimental Design

The modeling stage consumes the processed development and test files produced by the upstream data pipeline. For each of the three datasets, the data pipeline creates a stratified 80/20 development–test split. The test partition is held out during model comparison and is used only after cross-validation has been completed. All three candidate algorithms receive the same processed feature matrix for a given dataset so that differences in results are attributable to the model rather than to different input representations.

Within the development partition, we apply stratified five-fold cross-validation with shuffling and a fixed random seed of 42. Stratification preserves approximately the same class proportion in each fold, which is particularly important when the positive healthcare outcome is less frequent [9]. The exact same fold-generation rule and random seed are used for AdaBoost, XGBoost, and LightGBM. For every fold, four folds are used for model fitting and the remaining fold is used for validation. Model objects are re-created from scratch for every fold to prevent fitted state from leaking between folds.

To reduce the effect of class imbalance during fitting, balanced sample weights are computed from the training portion of each fold only. Validation and test sets remain untouched so that the reported metrics reflect the natural class prevalence of each dataset. A fixed probability threshold of 0.50 is used for the threshold-dependent metrics. No threshold is optimized on the

validation folds, which avoids adding a second tuning procedure that could make fold comparisons less consistent.

V.5. Model Configuration and Hyperparameters

The core comparison uses one fixed, literature-guided configuration per algorithm across all three datasets. We intentionally do not select a different best hyperparameter configuration for each dataset using the same five folds that are later used to quantify stability. Doing so could make the reported cross-validation scores optimistic and would confound RQ2 with hyperparameter search variability. The selected settings use conservative learning rates, moderate ensemble sizes, and regularization/subsampling for the gradient-boosted models.

Model	Selected hyperparameters	Rationale
AdaBoost	Decision stump (max depth = 1); 200 estimators; learning rate = 0.10	Decision stumps are the standard weak learner for AdaBoost. The larger ensemble combined with a smaller learning rate provides gradual boosting while limiting the contribution of each individual stump.
XGBoost	300 estimators; learning rate = 0.05; max depth = 4; subsample = 0.80; column sample = 0.80; L_2 regularization = 1; histogram tree method	A low learning rate and moderate tree depth control model complexity. Row and column subsampling reduce variance, while L_2 regularization makes leaf weights more conservative. The histogram method improves efficiency on the larger datasets.
LightGBM	300 estimators; learning rate = 0.05; 31 leaves; max depth = 6; minimum child samples = 20; subsample = 0.80; column sample = 0.80; L_2 regularization = 1	The number of leaves and depth cap constrain the leaf-wise tree growth. The minimum child size, subsampling, and regularization reduce overfitting while the small learning rate supports a smoother boosting process. Deterministic CPU settings and a fixed seed are enabled for reproducibility.

Table 4: Fixed model configurations used for the cross-dataset comparison.

V.6. Evaluation Metrics and Model Comparison

Each validation fold is evaluated using ROC-AUC, PR-AUC, Recall, and F1-score. ROC-AUC measures ranking performance across classification thresholds. PR-AUC focuses on the precision-recall trade-off and is especially informative when the positive class is relatively rare [10]. Recall measures the proportion of positive cases correctly identified, while F1-score balances precision and recall at the fixed 0.50 threshold. Because the three datasets may have different class prevalence, PR-AUC is used as the primary ranking metric, with ROC-AUC, F1-score, and fold variability retained for a more complete comparison.

For RQ1, we report the mean five-fold score of every metric for every model-dataset pair and rank the three models within each dataset by mean PR-AUC. We additionally report the average within-dataset rank across all three datasets rather than pooling patient rows across datasets. This prevents the largest dataset from dominating the overall comparison.

For RQ2, predictive stability is measured directly from the five fold-level scores. For each metric we report the mean, standard deviation, minimum, maximum, and range across folds. A smaller standard deviation and range indicate that the model is less sensitive to the particular development-data partition. After cross-validation, each model is re-fitted once on the full development partition and evaluated on the untouched test partition. These test results are reported separately from the cross-validation results and are not used to select the best model.

The complete experiment configuration is stored in `config.yaml`. The implementation writes fold-level metrics, aggregated cross-validation summaries, held-out test metrics, per-dataset rankings, cross-dataset rankings, and an experiment metadata file containing package versions and a hash of the configuration. This makes the model experiments repeatable and provides the fold-level outputs required by the later SHAP stability analysis.

V.7. SHAP Explainability and Feature-Ranking Stability (RQ3)

To address RQ3, this study uses SHAP (SHapley Additive exPlanations) to quantify how strongly each input feature contributes to model predictions. SHAP assigns an additive attribution value to each feature for an individual prediction and is grounded in Shapley-value principles from cooperative game theory [4]. For tree ensembles, TreeSHAP provides an efficient algorithm for obtaining these attributions while preserving the consistency properties of SHAP [5].

The explainability analysis follows the same stratified five-fold cross-validation partitions used for predictive evaluation. For each dataset, model, and fold, the model is fitted only on the training portion of that fold. SHAP values are then computed on a deterministic sample of observations from the corresponding validation portion. This design keeps the explained observations separate from the rows used to fit the model and allows changes in feature importance to be measured as the training sample changes across folds. The random seed, maximum number of explained validation observations, and other SHAP settings are stored in `config.yaml` so that the analysis is reproducible.

For XGBoost and LightGBM, the implementation uses TreeSHAP through `TreeExplainer`. The scikit-learn AdaBoost classifier used in this study is not supported by TreeSHAP in the pinned SHAP implementation, so AdaBoost is explained using model-agnostic permutation SHAP with a deterministic background sample drawn from the fold-training data. This difference is treated as a computational implementation detail: the stability analysis compares feature rankings *within the same model across folds*, rather than comparing raw SHAP magnitudes between different explainer algorithms.

For a feature j in fold f , global importance is defined as the mean absolute SHAP value across the N_f explained validation observations:

$$I_{j,f} = \frac{1}{N_f} \sum_{i=1}^{N_f} |\phi_{i,j,f}|, \quad (1)$$

where $\phi_{i,j,f}$ is the SHAP value for feature j and observation i in fold f . Features are sorted in descending order of $I_{j,f}$ to produce one feature-importance ranking for every model and fold. Mean absolute SHAP is used because positive and negative contributions should not cancel when estimating global importance.

Ranking stability is evaluated in two complementary ways. First, Spearman rank correlation is computed for every pair of cross-validation folds using the full feature-ranking vectors. With five folds, this yields ten pairwise comparisons for each dataset-model combination. A Spearman coefficient closer to one indicates that the relative ordering of features changes little between folds. Second, top- k overlap is measured with the Jaccard index:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad (2)$$

where A and B are the sets of the top- k features from two folds. The experiment uses $k = 10$ as configured in `config.yaml`. The Spearman measure evaluates stability over the complete ranking, whereas top- k Jaccard overlap focuses on whether the most influential features remain consistent.

The implementation stores fold-level mean absolute SHAP values and ranks, pairwise fold-stability measurements, aggregated stability summaries, and a consensus feature ranking. The consensus table reports each feature’s average rank, rank standard deviation, mean absolute SHAP value, and frequency of appearing among the top- k features across folds. These outputs provide the evidence required to answer RQ3 without relying on a single fitted model or a single data split.

The corresponding empirical stability results are presented in Section VI.3. and summarized in Table 8. This separation keeps the present subsection focused on how the explanations are computed, while the Results and Discussion section evaluates how stable those explanations are in practice.

VI. Results and Discussion

VI.1. Predictive Performance Across Datasets (RQ1)

Table 5 summarizes the five-fold cross-validation performance of AdaBoost, XGBoost, and LightGBM on the three healthcare datasets. The values are reported as mean $\pm$ standard deviation across the five validation folds. PR-AUC is treated as the primary comparison metric, while ROC-AUC, Recall, and F1-score provide complementary views of ranking and threshold-based performance.

Dataset	Model	ROC-AUC	PR-AUC	Recall	F1
Dataset 1	AdaBoost	0.8716 $\pm$ 0.0042	0.3853 $\pm$ 0.0087	0.6794 $\pm$ 0.0106	0.3765 $\pm$ 0.0082
	XGBoost	0.8872 $\pm$ 0.0033	0.4138 $\pm$ 0.0084	0.7782 $\pm$ 0.0071	0.3324 $\pm$ 0.0026
	LightGBM	0.8873 $\pm$ 0.0033	0.4129 $\pm$ 0.0088	0.7771 $\pm$ 0.0084	0.3346 $\pm$ 0.0031
Dataset 2	AdaBoost	0.9241 $\pm$ 0.0419	0.9226 $\pm$ 0.0444	0.8449 $\pm$ 0.0294	0.8578 $\pm$ 0.0190
	XGBoost	0.9244 $\pm$ 0.0322	0.9214 $\pm$ 0.0324	0.8815 $\pm$ 0.0381	0.8785 $\pm$ 0.0249
	LightGBM	0.9241 $\pm$ 0.0333	0.9152 $\pm$ 0.0351	0.8769 $\pm$ 0.0366	0.8726 $\pm$ 0.0256
Dataset 3	AdaBoost	0.8324 $\pm$ 0.0030	0.3681 $\pm$ 0.0074	0.7743 $\pm$ 0.0105	0.3825 $\pm$ 0.0027
	XGBoost	0.8386 $\pm$ 0.0027	0.3800 $\pm$ 0.0053	0.8016 $\pm$ 0.0049	0.3814 $\pm$ 0.0020
	LightGBM	0.8381 $\pm$ 0.0026	0.3801 $\pm$ 0.0057	0.7970 $\pm$ 0.0053	0.3825 $\pm$ 0.0021

Table 5: Five-fold cross-validation performance (mean $\pm$ standard deviation).

On Dataset 1, XGBoost and LightGBM produced almost identical ROC-AUC, while XGBoost achieved a slightly higher mean PR-AUC and Recall. Both gradient-boosted models clearly

exceeded AdaBoost on ROC-AUC, PR-AUC, and Recall. The lower F1 scores of XGBoost and LightGBM indicate that their higher recall at the fixed 0.50 threshold came with a precision trade-off.

On Dataset 2, all three models achieved strong discrimination, with mean ROC-AUC values close to 0.924. XGBoost achieved the highest mean Recall and F1, whereas AdaBoost obtained the highest mean PR-AUC by a small margin. This shows that no single model dominated every metric on this relatively small dataset.

On Dataset 3, XGBoost and LightGBM again improved ROC-AUC and PR-AUC over AdaBoost. XGBoost achieved the highest mean Recall, while the three models had very similar F1 values. Across the three datasets, the aggregated comparison ranked XGBoost first with a mean dataset rank of 1.6667, followed by LightGBM at 2.0000 and AdaBoost at 2.3333. XGBoost also achieved the highest cross-dataset mean ROC-AUC (0.8834), PR-AUC (0.5717), and Recall (0.8205). AdaBoost achieved the highest cross-dataset mean F1 (0.5389), emphasizing that the preferred model depends partly on the evaluation objective and decision threshold.

Model	Mean Rank	ROC-AUC	PR-AUC	Recall	F1
XGBoost	1.6667	0.8834	0.5717	0.8205	0.5308
LightGBM	2.0000	0.8832	0.5694	0.8170	0.5299
AdaBoost	2.3333	0.8760	0.5587	0.7662	0.5389

Table 6: Unweighted cross-dataset comparison based on the experiment outputs.

These results answer RQ1 by showing that XGBoost provides the strongest overall performance under the selected protocol, although LightGBM is extremely close on several ranking metrics and AdaBoost retains a small advantage in aggregate F1. Therefore, the comparison does not support a claim that one algorithm is uniformly superior on every metric and dataset.

Dataset	Model	ROC-AUC	PR-AUC	Recall	F1
Dataset 1	AdaBoost	0.8695	0.3860	0.6705	0.3657
	XGBoost	0.8850	0.4169	0.7712	0.3255
	LightGBM	0.8850	0.4187	0.7694	0.3259
Dataset 2	AdaBoost	0.9256	0.9394	0.8529	0.8832
	XGBoost	0.9224	0.9216	0.8333	0.8586
	LightGBM	0.9189	0.9166	0.8431	0.8643
Dataset 3	AdaBoost	0.8326	0.3672	0.7674	0.3788
	XGBoost	0.8384	0.3756	0.7963	0.3773
	LightGBM	0.8385	0.3763	0.7925	0.3789

Table 7: Performance on the untouched hold-out test partitions after refitting on the full development data.

VI.2. Predictive Stability Across Cross-Validation Folds (RQ2)

The fold-level results also reveal clear differences in stability across the three datasets. Dataset 1 and Dataset 3 produced small standard deviations for all three models. For example, XGBoost obtained ROC-AUC standard deviations of 0.0033 on Dataset 1 and 0.0027 on Dataset 3. LightGBM showed similarly small variation, indicating that the two gradient-boosted models were not highly sensitive to the particular fold assignment on the two larger datasets.

Dataset 2 exhibited noticeably greater fold-to-fold variation. Its development partition contains only 734 samples, compared with 353,653 for Dataset 1 and 183,824 for Dataset 3. XGBoost ROC-AUC ranged from 0.8676 to 0.9465 across the five folds, while AdaBoost ranged from 0.8547 to 0.9673 and LightGBM ranged from 0.8651 to 0.9497. The larger variability is therefore consistent with the smaller sample size: each validation fold contains fewer observations, so the composition of a single fold has a greater influence on the measured score.

The final hold-out results were generally close to the corresponding cross-validation means. For example, Dataset 1 XGBoost achieved a mean CV ROC-AUC of 0.8872 and a hold-out ROC-AUC of 0.8850, while Dataset 3 XGBoost achieved 0.8386 in CV and 0.8384 on the hold-out set. This agreement provides additional evidence that the reported cross-validation estimates are not driven by a single unusually favorable split.

Overall, RQ2 is answered by finding that predictive performance is highly stable on the two large datasets but less stable on the small Heart Failure Prediction dataset. The pattern is observed across all three algorithms, suggesting that dataset size and fold composition contribute more to the observed instability than the choice among the three boosting methods alone.

VI.3. SHAP Feature-Importance Stability Across Cross-Validation Folds (RQ3)

Following the fold-wise explainability procedure described in Section V.7., RQ3 evaluates whether the SHAP-based global feature-importance rankings remain consistent when the development data are repartitioned across cross-validation folds. For each dataset-model pair, all ten pairwise combinations of the five folds are compared using Spearman rank correlation over the complete feature ranking and top-10 Jaccard similarity over the most important features.

Table 8 shows that the SHAP rankings are highly stable across folds for all nine dataset-model combinations. Mean Spearman correlations range from 0.9332 to 0.9700, indicating that the overall ordering of features changes only modestly when the training and validation samples are changed. Mean top-10 Jaccard similarity ranges from 0.7455 to 1.0000, showing that the most influential features are also largely preserved across folds.

Dataset	Model	Spearman ρ	Std.	Top-10 Jaccard	Std.
Dataset 1	AdaBoost	0.9652	0.0179	0.9273	0.0939
	XGBoost	0.9332	0.0114	0.8545	0.0767
	LightGBM	0.9479	0.0053	0.7455	0.1145
Dataset 2	AdaBoost	0.9467	0.0226	0.8061	0.0928
	XGBoost	0.9389	0.0217	0.8727	0.0878
	LightGBM	0.9348	0.0251	0.8364	0.0575
Dataset 3	AdaBoost	0.9599	0.0277	1.0000	0.0000
	XGBoost	0.9686	0.0162	1.0000	0.0000
	LightGBM	0.9700	0.0130	0.8727	0.0878

Table 8: SHAP feature-ranking stability across the ten pairwise comparisons of five cross-validation folds. Values are pairwise means and standard deviations.

Dataset 3 exhibits the strongest top-feature stability. AdaBoost and XGBoost both achieve a mean top-10 Jaccard similarity of 1.0000, meaning that the same set of ten features appears in the top-10 for every fold pair. LightGBM also shows very high full-ranking stability on this dataset, with the highest mean Spearman correlation of 0.9700. The consensus rankings are also

strongly aligned across the three models: **Age**, **GenHlth**, **Sex**, **HighBP**, and **HighChol** occupy the leading positions for all three algorithms.

On Dataset 2, all models remain stable despite the substantially smaller sample size observed in the predictive-stability analysis. Mean Spearman correlations remain above 0.93 and mean top-10 Jaccard similarity ranges from 0.8061 to 0.8727. Across the three consensus rankings, **ST_Slope_Up** and **ChestPainType_ASY** are consistently ranked first and second, while features such as **Sex** and **ST_Slope_Flat** also appear near the top for multiple models.

Dataset 1 also shows strong agreement in the complete rankings, with mean Spearman correlations between 0.9332 and 0.9652. AdaBoost has the highest mean top-10 overlap (0.9273), while LightGBM has a lower top-10 Jaccard value (0.7455) despite a high mean Spearman correlation of 0.9479. This combination suggests that LightGBM’s overall feature ordering is stable, but a small number of features close to the top-10 boundary exchange positions across folds. The consensus rankings nevertheless show substantial cross-model agreement: **HadAngina**, **ChestScan**, **Sex**, and **RemovedTeeth_None_of_them** are among the leading features for all three models.

These results answer RQ3 by showing that SHAP-based global feature-importance rankings are generally robust to cross-validation fold assignment. The evidence is consistent across two complementary stability measures: full-ranking correlation remains high, while the set of highest-ranked features is usually well preserved. At the same time, the lower top-10 overlap observed for some model–dataset combinations shows why reporting both measures is useful: a high correlation over the full feature set does not necessarily imply an identical membership of the top-10 features.

VI.4. Discussion

Taken together, the three research questions provide complementary evidence about predictive quality and interpretability. RQ1 shows that XGBoost provides the strongest overall predictive ranking across the three datasets, with LightGBM very close on ROC-AUC and PR-AUC and AdaBoost retaining the highest cross-dataset mean F1. RQ2 shows that predictive scores are highly stable on the two large datasets but vary more on the smaller Heart Failure Prediction dataset. RQ3 extends the stability analysis from prediction metrics to model explanations and finds that the SHAP feature rankings remain highly consistent across folds for all three boosting methods.

The SHAP results should be interpreted as measures of explanation stability, not as evidence of causality. A feature can be consistently important to a model because it is predictive under the observed data distribution, but this does not establish that changing the feature would causally change a patient’s risk. In addition, raw SHAP magnitudes are not compared directly between AdaBoost and the two gradient-boosted models because AdaBoost is explained with permutation SHAP whereas XGBoost and LightGBM use TreeSHAP. The study therefore bases RQ3 on within-model fold-to-fold ranking consistency, which is comparable across the experimental protocol.

Overall, the combined findings indicate that the strongest models are not only competitive in predictive performance but also produce feature-importance patterns that are reproducible across data partitions. This is useful for healthcare risk-prediction studies because an explanation that changes sharply with a different fold would be difficult to trust even if average predictive per-

formance were high. The observed stability therefore provides an additional criterion, alongside ROC-AUC, PR-AUC, Recall, and F1, for assessing the robustness of the evaluated models.

VII. Conclusion and Future Work

VII.1. Conclusion

VIII. References

- [1] Y. Freund et al., “A decision-theoretic generalization of on-line learning and an application to boosting”, *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997. DOI: 10.1006/jcss.1997.1504.
- [2] T. Chen et al., “XGBoost: A scalable tree boosting system”, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785.
- [3] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree”, in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.
- [4] S. M. Lundberg et al., “A unified approach to interpreting model predictions”, in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] S. M. Lundberg et al., “From local explanations to global understanding with explainable ai for trees”, *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020. DOI: 10.1038/s42256-019-0138-9.
- [6] K. Pytlak, *Personal key indicators of heart disease (2022 update)*, Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/kamilpytlak/personal-key-indicators-of-heart-disease>.
- [7] Fedesoriano, *Heart failure prediction dataset*, Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction>.
- [8] A. Teboul, *Heart disease health indicators dataset (brfss 2015)*, Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/alexteboul/heart-disease-health-indicators-dataset>.
- [9] F. Pedregosa et al., “Scikit-learn: Machine learning in python”, *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] T. Saito et al., “The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets”, *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. DOI: 10.1371/journal.pone.0118432.

Appendix

1. **AIVIETNAM:** Further information about AIVIETNAM can be found [here](#).
2. **GitHub Repository:**
3. **Dataset:**
4. **Demo Video / Presentation:**
 - Link demo video: [Link demo](#)
 - Link slide presentation: [Link slide presentation](#)